Java OOP(Object Oriented Programming) Concepts
Last Updated : 8 Apr, 2026

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects that contain data (fields) and behavior (methods). It focuses on designing software that closely represents real-world entities. It is used to:

- Improves code reusability
- Enhances maintainability and scalability
- Makes programs easier to understand and manage
- Closely models real-world entities

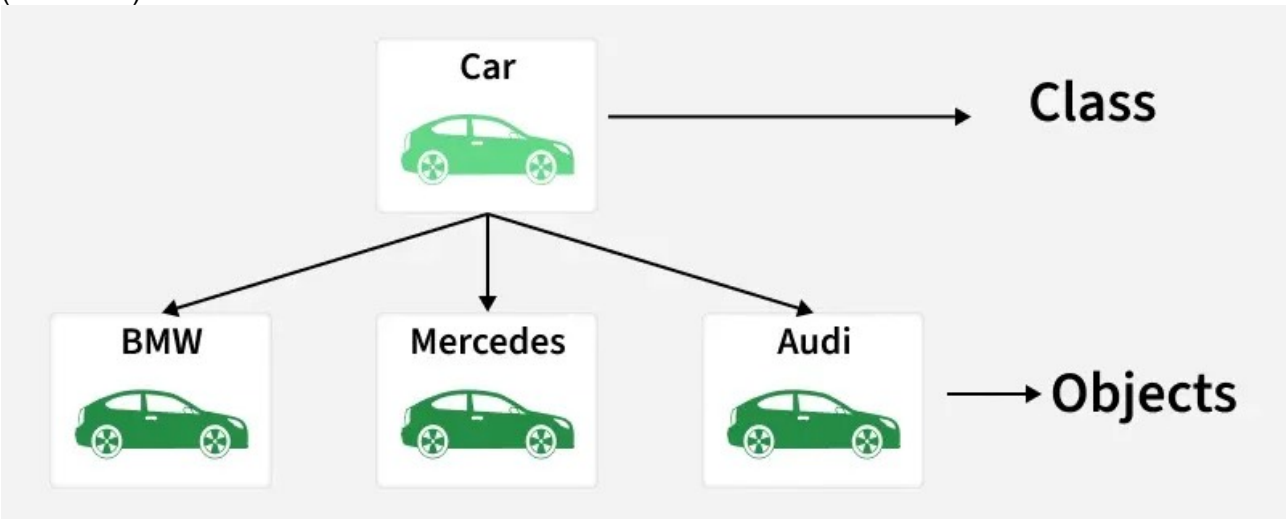
Characteristics of an OOP(Object Oriented Programming)
The diagram below demonstrates the Java OOPs Concepts

OOP's concept

Class
A Class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times. In general, class declarations can include these components in order:

- Modifiers: A class can be public or have default access (Refer to this for details).
- Class name: The class name should begin with the initial letter capitalized by convention.
- Body: The class body is surrounded by braces, { }.

Example: A Car represents a class (blueprint), while BMW, Mercedes, and Audi represent objects (instances) created from that class.

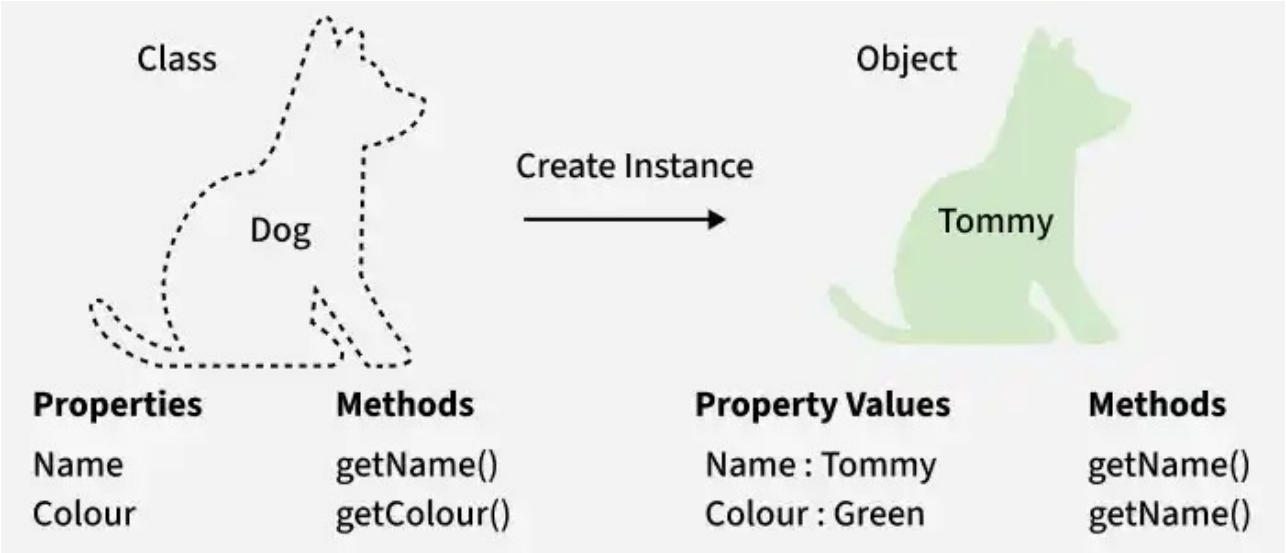


class-object

Object
An Object is a basic unit of Object-Oriented Programming that represents real-life entities. A typical Java program creates many objects, which as you know, interact by invoking methods. The objects are what perform your code, they are the part of your code visible to the viewer/user. An object mainly consists of:

- State: It is represented by the attributes of an object. It also reflects the properties of an object.
- Behavior: It is represented by the methods of an object. It also reflects the response of an object to other objects.
- Identity: It is a unique name given to an object that enables it to interact with other objects.
- Method: A method is a collection of statements that perform some specific task and return the result to the caller.

Example: Dog is a class, Tommy is an object of that class.



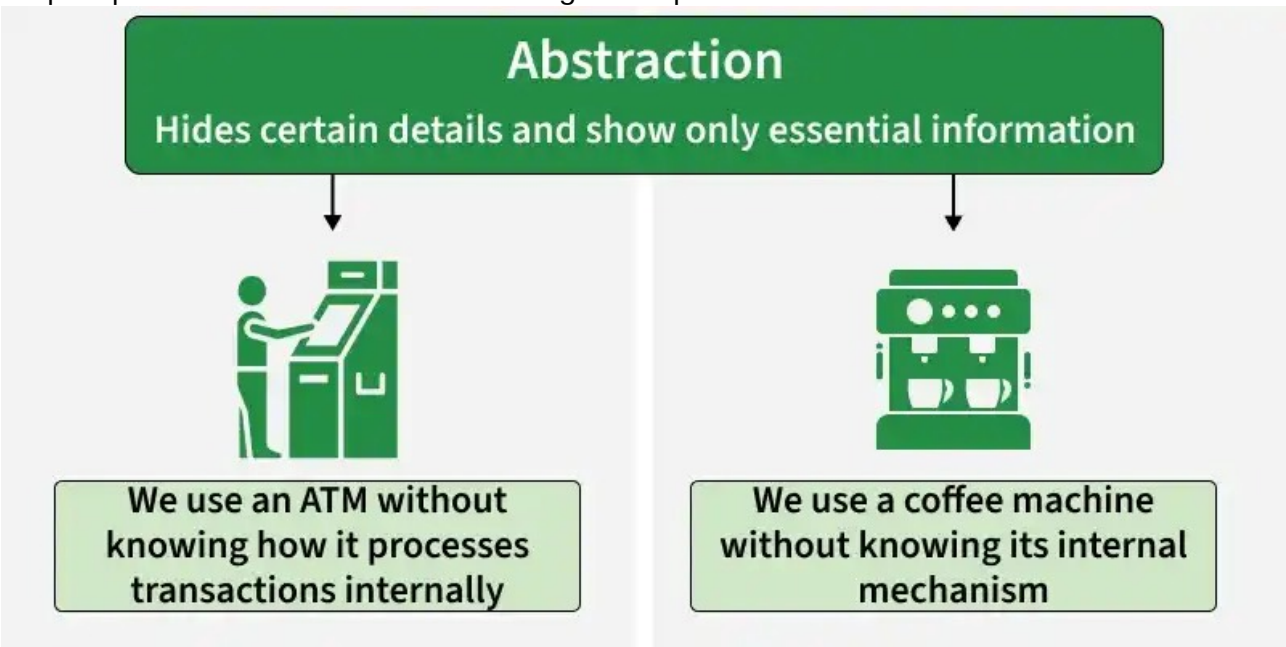
Object

Abstraction

Abstraction in Java is the process of hiding implementation details and showing only the essential features of an object. It helps users focus on what an object does rather than how it does it.

- Hides complexity: Internal implementation details are hidden from the user.
- Improves maintainability: Changes in implementation do not affect the user code.
- Enhances flexibility: Supports loose coupling through abstract classes and interfaces.

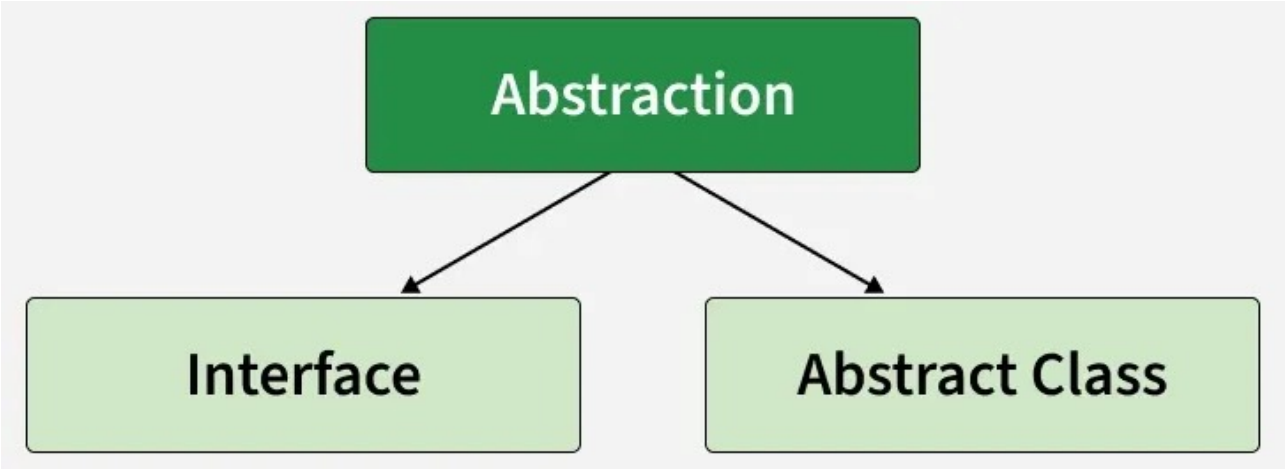
Example: An ATM or a coffee machine represents abstraction, where the user interacts with simple operations while the internal working and implementation details remain hidden.



Abstraction

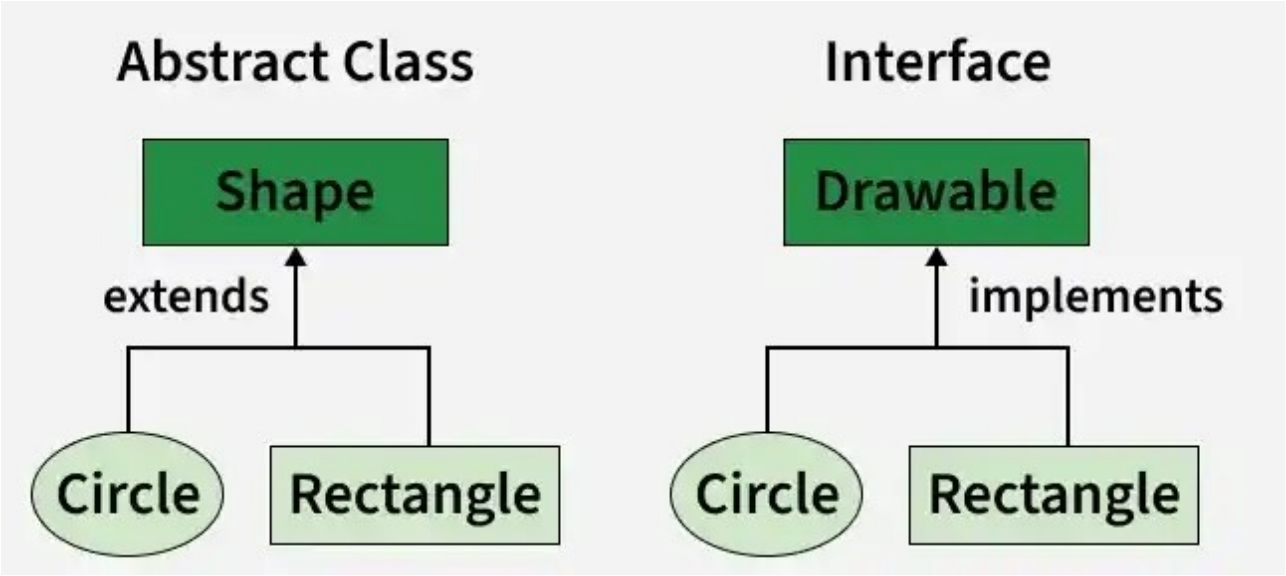
How to Achieve Abstraction

- It is achieved in Java using abstract classes and interfaces.
- Interfaces can provide 100% abstraction, while abstract classes provide partial abstraction.



Achieve Abstraction

Example: Abstract class defines a common base (Shape) using inheritance, while an interface (Drawable) defines behavior implemented by multiple classes.

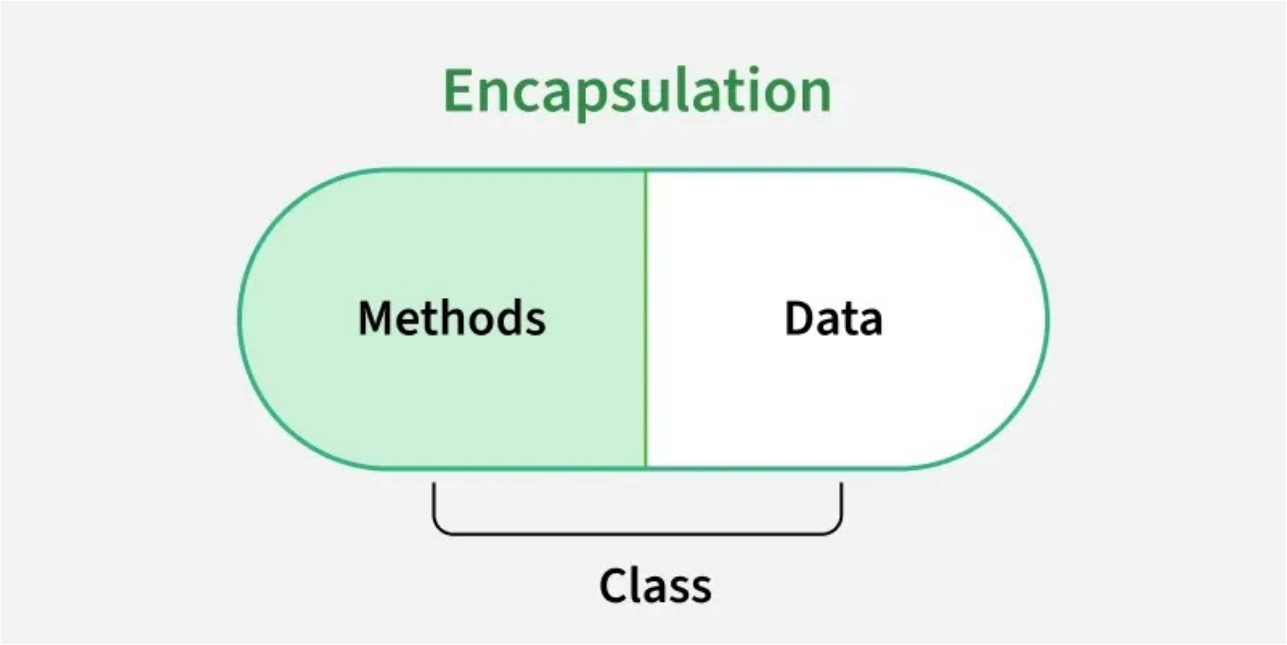


abstract-class-interface

Encapsulation

Encapsulation is the process of wrapping data and methods into a single unit, usually a class, and restricting direct access to the data. It acts as a protective shield that prevents data from being accessed directly from outside the class.

- Data members are hidden using the private access modifier.
- Access to data is provided through public getter and setter methods.
- It improves data security, maintainability, and controlled access.



Encapsulation

Association

Association is an OOP concept that defines a relationship between two or more classes that are connected to each other. It represents how objects interact with each other and communicate. In association, objects of one class are related to objects of another class, but they can exist independently.

- Represents a relationship between objects
- Does not imply ownership
- Objects are independent of each other

Types of Association

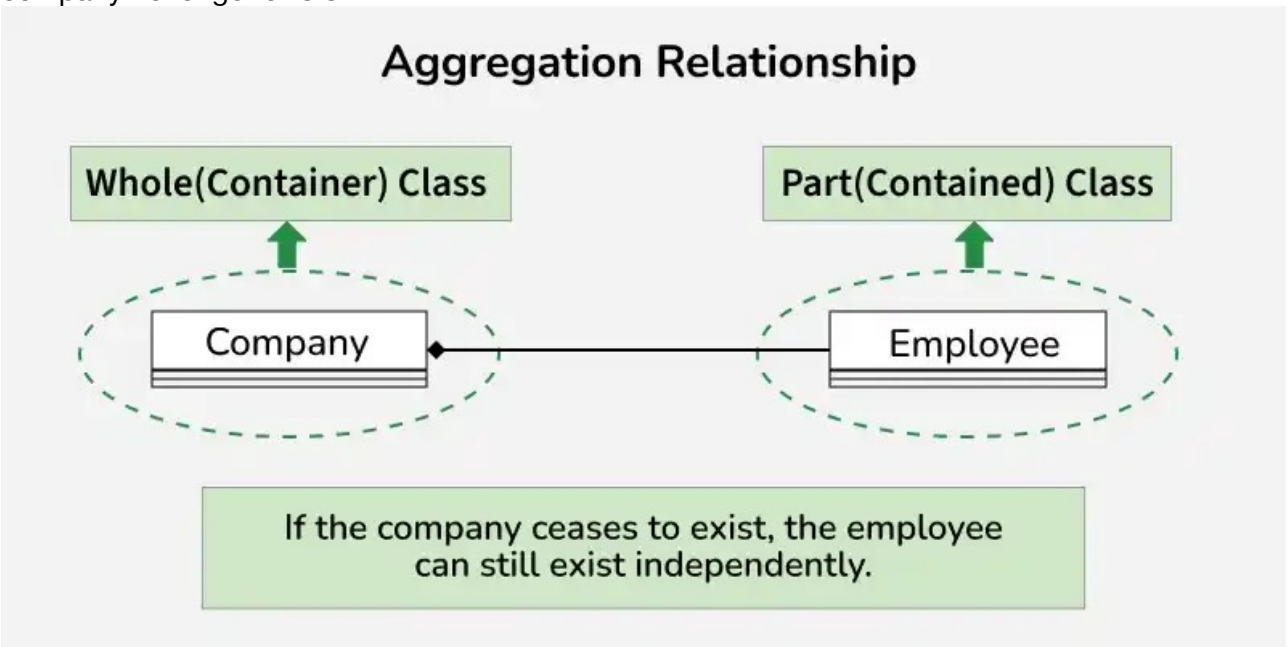
Association in Java can be further classified into the following types:

1. Aggregation (Weak Association)

Aggregation represents a “has-a” relationship where one class contains a reference to another class, but both can exist independently.

- It is a weak relationship
- Objects have independent lifecycles
- One object can exist without the other

Example: A Company has Employees, but employees can exist independently even if the company no longer exists.



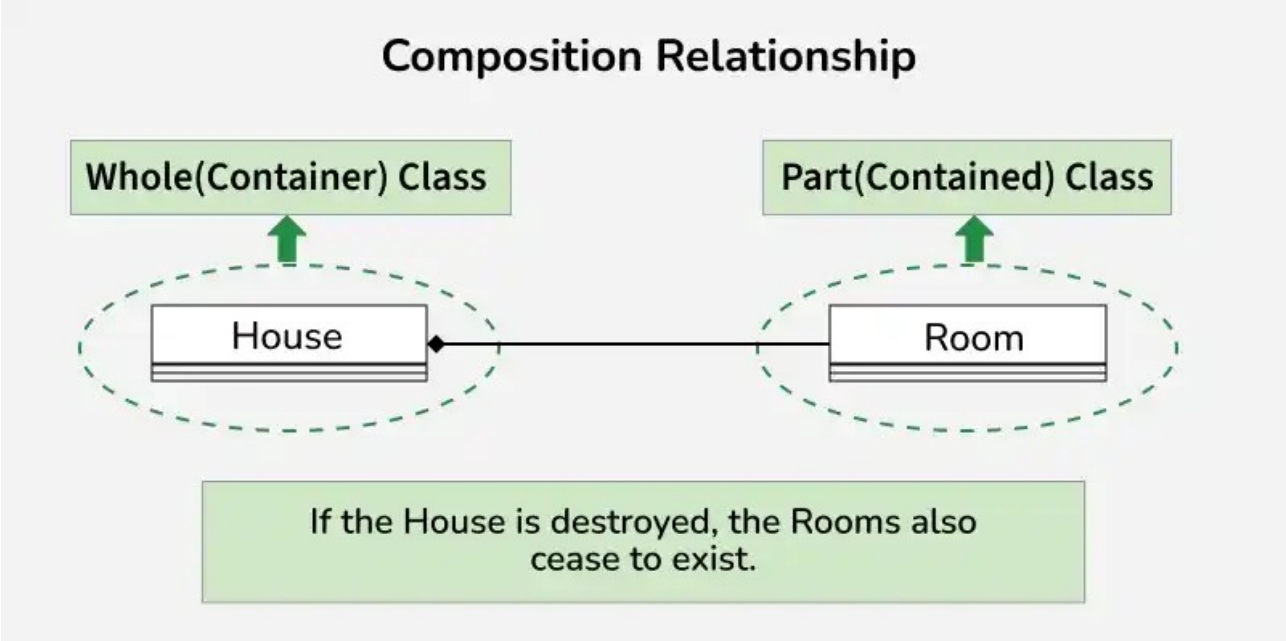
Aggregation

2. Composition (Strong Association)

Composition is a strong form of association where one class owns another class. If the parent object is destroyed, the child object also gets destroyed.

- It is a strong relationship
- Objects have dependent lifecycles
- Child object cannot exist without the parent

Example: A House is composed of Rooms, and if the house is destroyed, the rooms cannot exist independently.

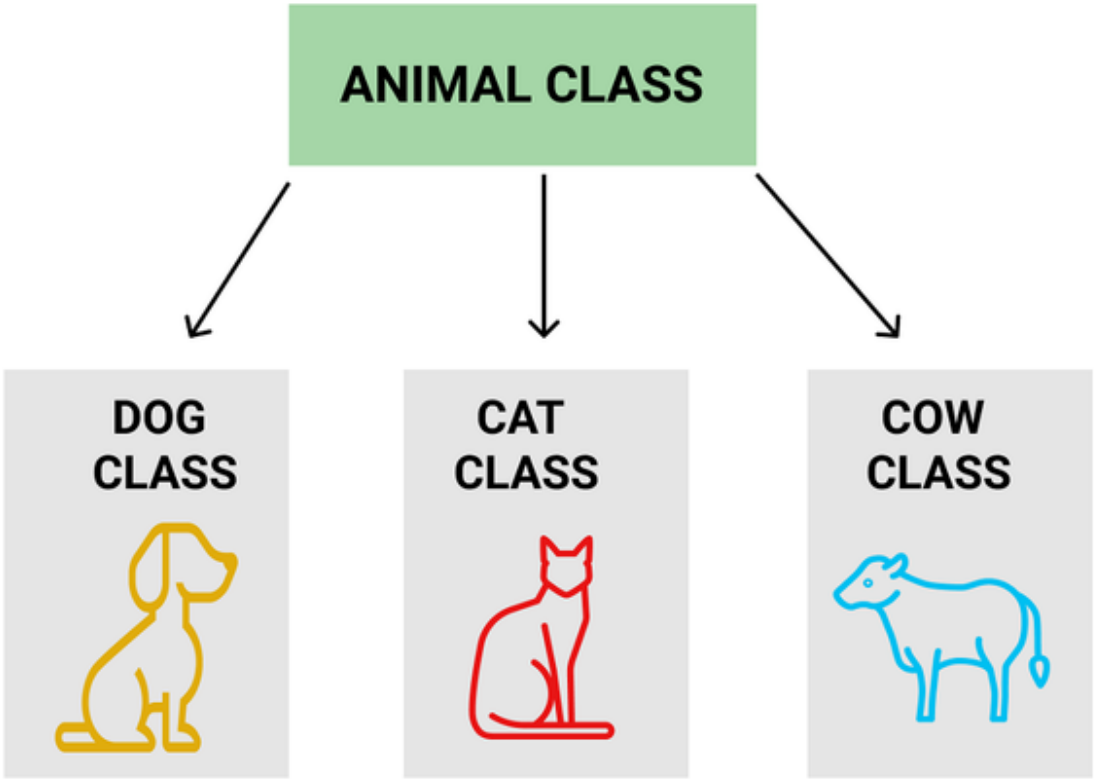


Inheritance

Inheritance is a core OOP concept in Java that allows one class to acquire the fields and methods of another class using the extends keyword. It represents an “is-a” relationship between classes.

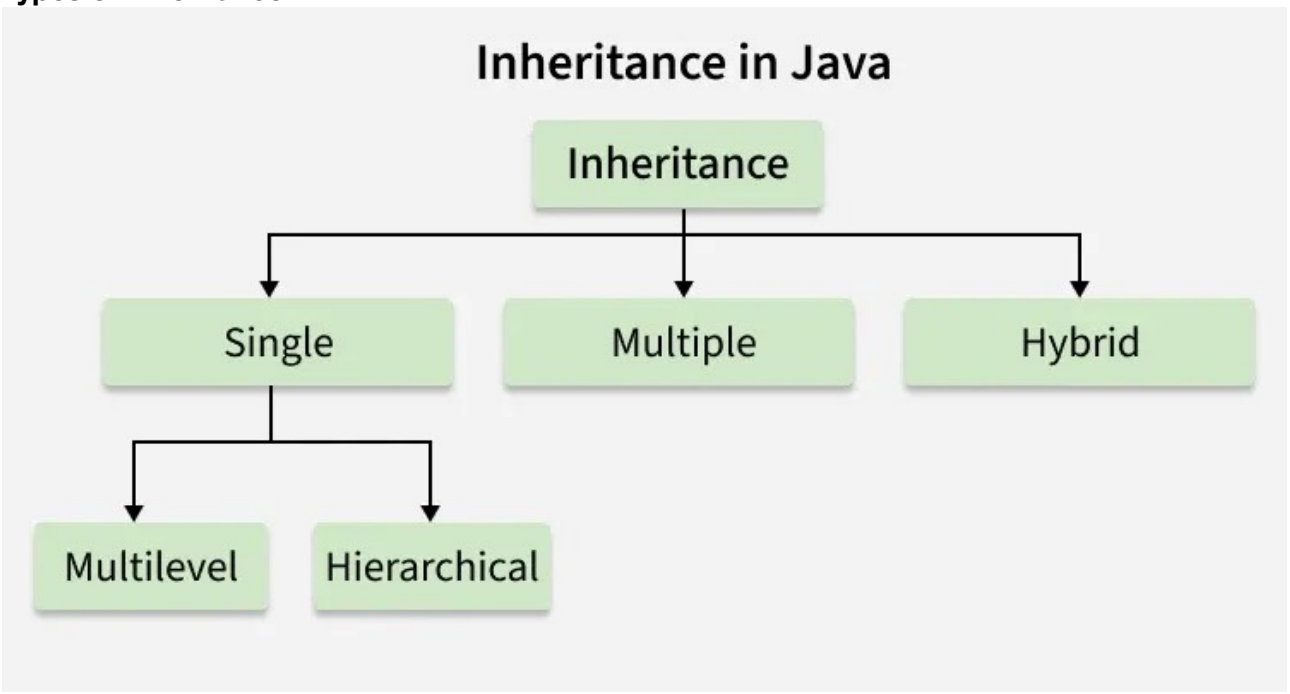
- The class being inherited is called the superclass, and the inheriting class is the subclass.
- A subclass can use existing features of the superclass and also add its own.
- Inheritance promotes code reusability and reduces redundancy.

Example: Dog, Cat, Cow can be Derived Class of Animal Base Class.



Inheritance

Types of Inheritance



Java supports the following types of inheritance:

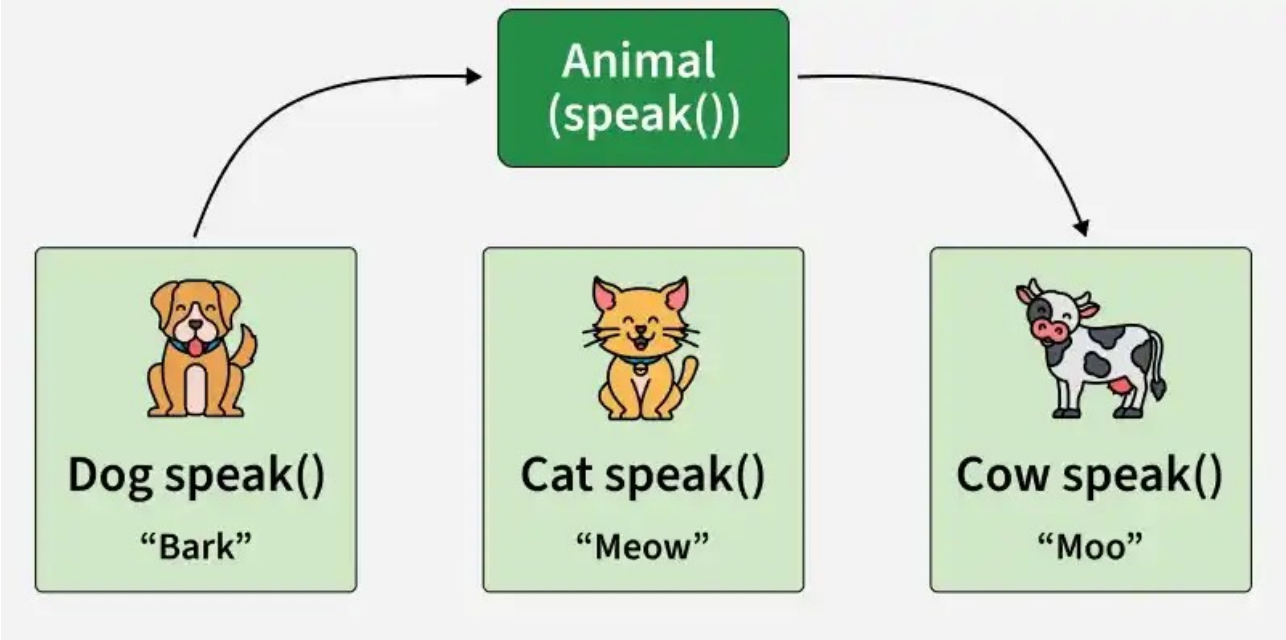
- Single Inheritance: One subclass inherits from one superclass.
- Multilevel Inheritance: A class is derived from another derived class, forming a chain.
- Hierarchical Inheritance: Multiple subclasses inherit from a single superclass.
- Multiple Inheritance (through Interface): A class inherits from multiple interfaces since Java does not support multiple inheritance using classes.
- Hybrid Inheritance (through Interface): A combination of two or more types of inheritance, achievable using interfaces.

Polymorphism

Polymorphism means “many forms”, where a single entity can behave differently in different situations. In Java, it allows the same method or object to show different behavior based on context.

- Same method, different behavior depending on the object
- Achieved through method overloading and method overriding

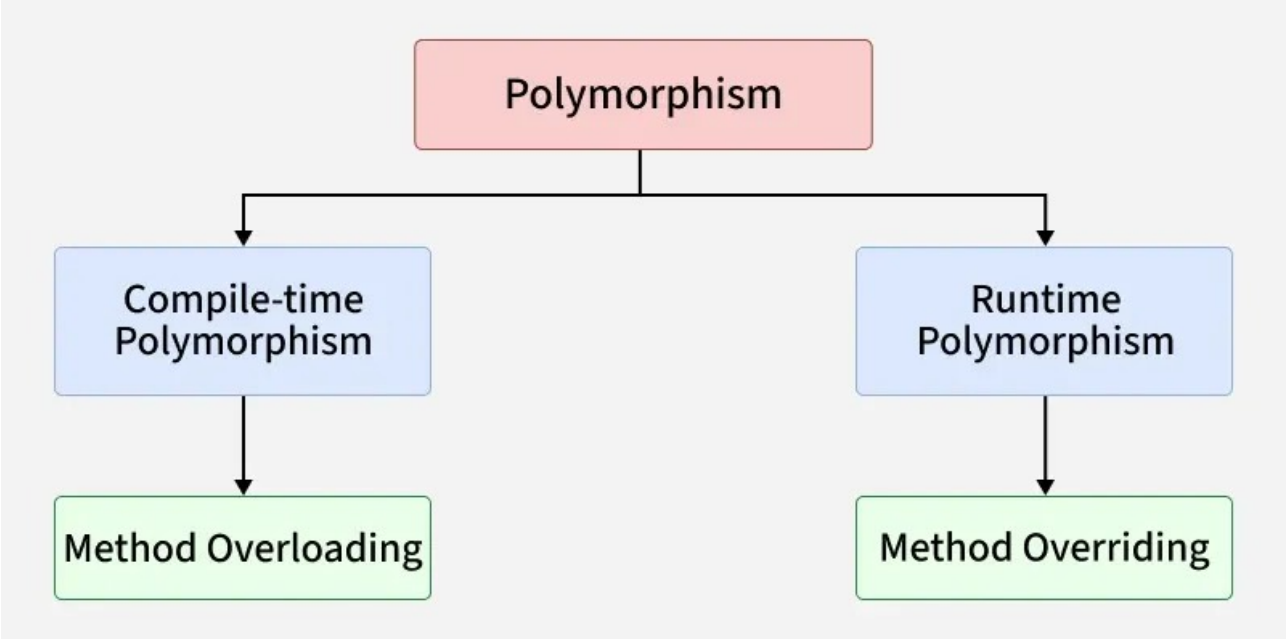
Example: Different animals represent polymorphism, where the same method speak() produces different outputs like Bark, Meow, and Moo depending on the object.



Polymorphism

Types of Polymorphism

Polymorphism in Java is mainly of 2 types as mentioned below:



Polymorphism in Java

- Compile-time Polymorphism(Method Overloading) :Achieved when multiple methods have the same name but different parameters. The method call is resolved at compile time.
- Runtime Polymorphism (Method Overriding): Achieved when a subclass provides a specific implementation of a method already defined in its superclass. The method call is resolved at runtime based on the object

Advantage of OOP over Procedure-Oriented Programming Language

Object-oriented programming (OOP) offers several key advantages over procedural programming:

- Code reusability: Classes and objects allow reuse of existing code, reducing duplication and improving efficiency.
- Better structure and maintainability: Programs are organized into logical units, making code easier to understand, debug, and maintain.
- Supports DRY principle: Common functionality is written once and reused, leading to cleaner and more maintainable code.
- Faster development: Modular and reusable components help in quicker and scalable application development.

Disadvantages of OOP

- Steep learning curve: Concepts like classes, objects, inheritance, and polymorphism can be difficult for beginners.
- Overhead for small programs: OOP may require more code and structure than necessary for simple applications.
- Debugging complexity: Code spread across multiple classes and layers can make debugging more time-consuming.
- Higher memory usage: Creating many objects can consume more memory compared to procedural programs.